

PROJECT BRIEF

vigil — 機台旁的視覺監看盒

邊緣錄影 + 雲端儲存 + 遠端儀表板，下一步：判斷「這台機器做的東西還對不對」

插電即開機、開機即錄影、錄影即切檔、切檔即上傳。

放在機器旁邊，看著它做完該做的事；做壞了，在還來得及的時候說。

專案位置：[repo_claude/vigil](#) (GitHub: MaxShih147/vigil)

文件版本：2026-09-05 • 依 README.md、docs/PRODUCT.md、docs/RESEARCH.md、pi/DEPLOY.md 與程式碼整理

對象：即將加入專案的工程同事（軟體 / 韌體 / 硬體 / 光學）

目錄

1. 一頁摘要
2. 專案是什麼、給誰用
3. 系統架構圖（部署拓撲）
4. 邊緣端元件架構圖（Raspberry Pi）
5. 現有流程：錄影 → 切檔 → 上傳 → 心跳 / 遠端指令
6. 雲端儀表板：登入授權流程與資料模型
7. 為什麼先做光固化 3D 列印、光學限制
8. 核心洞見與檢測階梯 L0-L4
9. 檢測管線流程圖（inspector 十個階段）
10. 列印工作狀態機
11. 失敗升級時序圖
12. API 與資料模型變更
13. 機台旁的螢幕（kiosk）
14. 階段路線圖與驗收條件
15. 風險與明確不做事
16. 開發上手指南（Mac 開發、Pi 部署）
17. 文件審閱：發現的不一致與交接前該處理的事

1. 一頁摘要

vigil 是一個放在機器旁邊的小盒子（Raspberry Pi 5 + USB webcam）。現在它已經會做「無聊的那一半」：開機就錄影、每 10 分鐘切一個 mp4、自動上傳到 Cloudflare R2、每 10 秒向雲端回報健康狀態、並接受遠端指令；儀表板架在 Vercel，用 Google 登入加白名單控管。接下來要做「值錢的那一半」：從影像判斷機器的工作是否正常，首個目標是 MSLA / 光固化樹脂 3D 列印的**脫板（detachment）偵測**。

項目	內容
已完成（Phase 1-5）	單機錄影、自動切檔、自動上傳 R2、heartbeat + 指令通道、Web 儀表板（Google OAuth、白名單、裝置控制、錄影瀏覽 / 下載）
下一步（Inspection）	Phase 0 光學可行性 → 週期鎖定 → L1 脫板偵測 + 事件管線 → kiosk 螢幕 + 通知 → 雲端判讀 → 切片比對 / 自動中止
設計核心約束	不接印表機、不需 API、不需廠商配合；盒子「看」機器的節奏自己學。Pi 在 NAT 後面，所有連線一律由 Pi 主動對外發起。
技術棧	邊緣：Python 3 + ffmpeg + boto3 + systemd；雲端：Next.js 16 / React 19 / Auth.js（Google） / Drizzle ORM / Neon Postgres / Cloudflare R2 / Vercel
第一個要回答的問題	Phase 0：相機 + 紅光透過橘色罩子，到底看不看得出樹脂列印在前 10% 層數就脫板？看不出來，軟體救不了，先改機構與打光。
為什麼值得做	一次樹脂列印跑 4-12 小時，脫板多發生在前 10%，印表機自己不知道，繼續空轉一整晚：耗樹脂、風險戳破 FEP、機時全丟、交期延一輪。十秒內抓到，一夜報廢變成 20 分鐘的重來。

三個角色，通常不在同一棟樓

角色	職責	通常在哪
Operator	把盒子裝在機器旁、供電、把鏡頭對好	現場、廠區
Consumer	看影片、看狀態；不碰裝置	任何辦公室、任何城市
Maintainer	擁有程式碼、雲端帳號、整個機隊	有筆電就行

因為三者很少在同一地點，影片與狀態一定要放雲端，不能放要 VPN / port-forward 才碰得到的 NAS。整個系統不假設有人能走過去看裝置。

2. 專案是什麼、給誰用

一個自給自足的盒子，放在機器旁邊，透過相機看機器工作，判斷工作是否正常，把判斷結果送到三個地方：雲端儀表板、機台旁的螢幕、通知管道。

它可以裝在完全不是為它設計的機器上：不整合機器的控制器、沒有 API、不需要廠商合作、沒有任何一條線接進機器——盒子靠「看」來學習機器的節奏。這個限制就是產品本身，第 8-9 章的所有設計都是為了守住這個承諾。

買家與使用情境

需要同時顧好多台印表機、但人力注意力無法等比放大的營運單位：牙科 / 矯正技工所、珠寶蠟模鑄造、模型公仔代印廠、打樣工作室、大學實驗室。八台機器的代印廠是理想的第一個客戶：價值隨機台數放大、注意力不會、而且規模小到不需要一年期的企業採購流程。

這明確不是 AOI。我們不是拿成品去對公差，我們回答一個更便宜也更值錢的問題：這一趟列印還值得繼續嗎？正是這個重新定義，讓一個盒子能用在沒見過的機器上，也讓它從困難的電腦視覺題目變成可解的題目。

Repo 結構

```
vigil/
├─ README.md           專案總覽、R2 設定、Mac 開發流程、Pi 部署
├─ docs/PRODUCT.md     產品與架構計畫 (Inspection 全部設計都在這裡, 492 行)
├─ pi/                 邊緣端程式 (跑在 Pi, 也能在 Mac 開發)
│   ├─ config/vigil.yaml 裝置 / 相機 / 錄影 / 上傳 / agent 設定
│   ├─ scripts/recorder.py 包 ffmpeg segment muxer
│   ├─ scripts/uploader.py 掃描穩定的 mp4 → 上傳 R2
│   ├─ scripts/agent.py   heartbeat、拉指令、執行、回報、清理
│   ├─ scripts/start_*.sh 啟動包裝 (建 venv、載入 .env)
│   ├─ systemd/*.service  vigil-recorder / vigil-uploader / vigil-agent
│   ├─ DEPLOY.md         Pi 5 從開箱到上線的 runbook (含 Tailscale)
│   └─ .env(.example)    R2 金鑰、儀表板 URL、裝置 API key (gitignored)
└─ web/                 Next.js 儀表板 (Vercel)
    ├─ src/auth.ts       Auth.js + Google, 白名單 / admin 判定
    ├─ src/proxy.ts       路由守門: 公開 / 需登入 / admin-only
    ├─ src/lib/db/schema.ts Drizzle schema (4 張表)
    ├─ src/app/api/devices/* Pi 呼叫的 bearer-token API
    ├─ src/app/{devices,admin,signin,request-access} 頁面
    └─ src/lib/{r2,recordings}.ts R2 列表與 presigned URL
```

3. 系統架構圖（部署拓撲）

四個區域，其中三個已經存在。決定一切的網路事實：**Pi 在工廠 NAT 後面，沒有 inbound 可達性**。所有邊緣 → 雲端互動都是 Pi 主動發起的 outbound 呼叫；沒有 push 到裝置的通道，遠端控制靠裝置輪詢指令佇列。另外，影片位元組由邊緣寫入 R2、由瀏覽器透過 presigned URL 直接讀取，不經過 Vercel functions（省 egress 費用）。

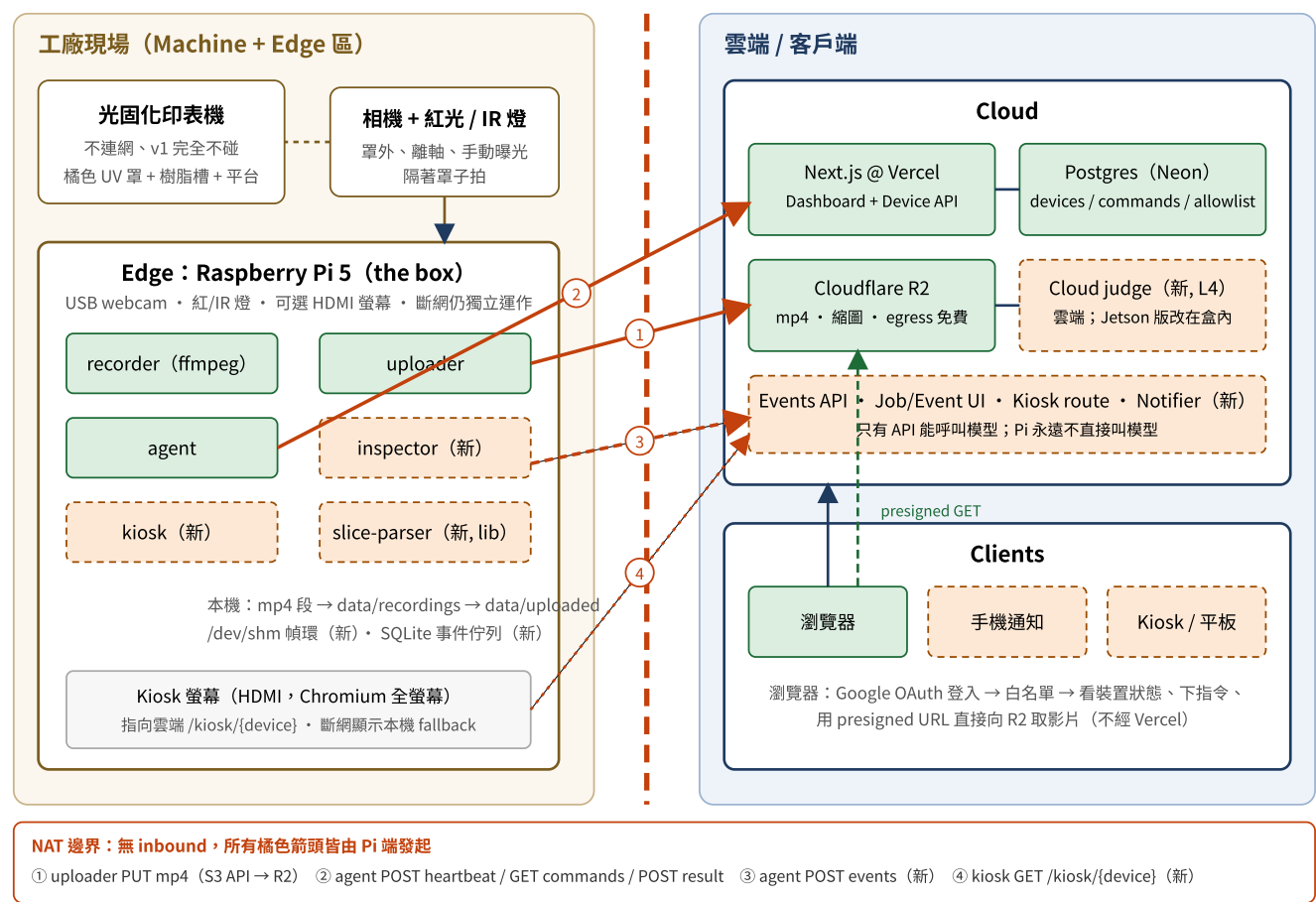


圖 1 部署拓撲。綠色實線框 = 已存在；橘色虛線框 = 規劃中。橘色箭頭全部由 Pi 端發起並穿越 NAT。

區域	內容	備註
Machine	樹脂印表機；相機與燈對著它	不與我們連網。v1 完全不碰它。
Edge (盒子)	Pi 5 (基本版) 或 Jetson Orin Nano (判讀版，可本地跑 L4)、USB webcam、紅光/IR、可選 HDMI 螢幕；四個服務	在工廠 NAT 後；斷網時獨立運作
Cloud	Vercel 上的 Next.js、Neon Postgres、Cloudflare R2、雲端判讀模型	
Clients	瀏覽器 (儀表板)、機台旁 kiosk、手機通知	

4. 邊緣端元件架構圖 (Raspberry Pi)

邊緣端是多個獨立程序，各自可單獨重啟（systemd 各一個 unit）。真正新的架構問題只有一個：**相機所有權**。USB webcam 不能被兩個程序同時開啟；`recorder` 已經握著它，而 `inspector` 需要幀。

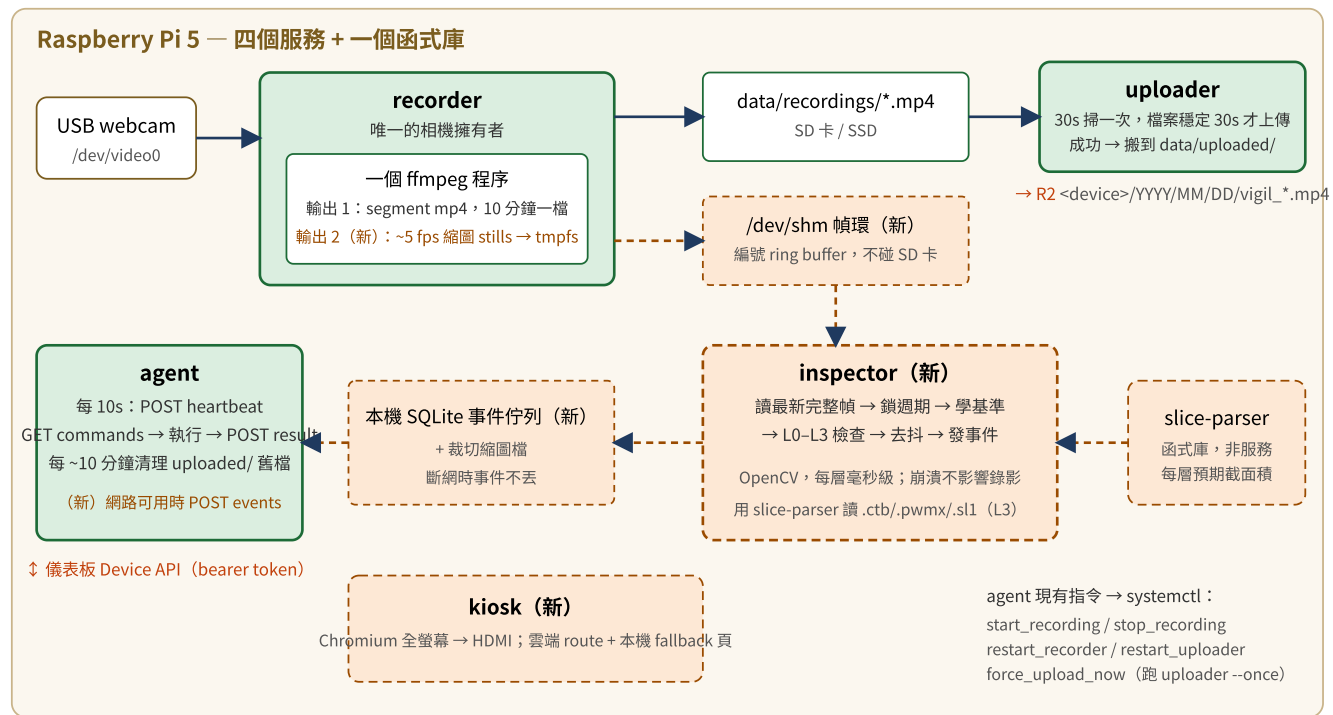


圖 2 邊緣端元件。核心設計：ffmpeg 維持唯一相機擁有者，tee 出第二路低幀率輸出到 tmpfs 幀環給 inspector。

相機所有權：考慮過的三個選項

方案	結論
1. inspector 擁有相機、自己重新編碼影片	❌ 把兩個乾淨的服務揉成一個，錄影路徑會被檢測 bug 拖累
2. v4l2loopback 把裝置扇出	❌ 每台盒子都要裝 out-of-tree kernel module，機隊維護負擔
3. ffmpeg 維持單一擁有者，tee 第二路輸出	✅ 各服務職責不變、無 kernel 依賴、inspector 崩潰不影響錄影、成本只有一個 ffmpeg output filter

第二路輸出刻意低幀率（~5 fps、縮小），寫到 `/dev/shm` 的小型編號 ring buffer，永不碰 SD 卡。週期偵測要解析的是數秒的抬升行程，不是動態模糊。**Phase 1 待確認**：幀環在負載下讀取無撕裂；5 fps 能在真機上分辨抬升 / 停留的轉換。

元件清單

元件	狀態	職責
recorder	存在	擁有相機。一個 ffmpeg 程序，依平台選 avfoundation (Mac) 或 v4l2 (Linux)；segment muxer 切 10 分鐘 mp4，libx264 veryfast，無音訊。

uploader	存在	掃描完成的 mp4，上傳到 S3 相容儲存（R2），本機搬到 uploaded/ 或刪除；失敗重試。
agent	存在	每 10 秒回報健康（是否錄影中、磁碟、待上傳數、IP、版本）；拉取並執行指令、回報結果；本機磁碟清理（保留 3 天）。
inspector	新	判斷引擎。吃幀、鎖機器週期、學基準、跑 L0-L3、把事件丟進本機佇列。
slice-parser	新	函式庫。解析 .ctb / .pwmx / .sl1，取得每層預期截面積供 L3 使用。
kiosk	新	Pi HDMI 上的 Chromium kiosk 模式，指向雲端 route，帶本機 fallback 頁。

5. 現有流程：錄影 → 切檔 → 上傳 → 心跳 / 遠端指令

這是今天已經在跑的東西（在 `vigil-dev-01` 實測過）。分成兩條互不干擾的流程：影片資料流，以及狀態 / 控制流。

A. 影片資料流（recorder → uploader → R2 → 瀏覽器）



B. 狀態與控制流（agent 每 10 秒一輪，全部 Pi 主動發起）

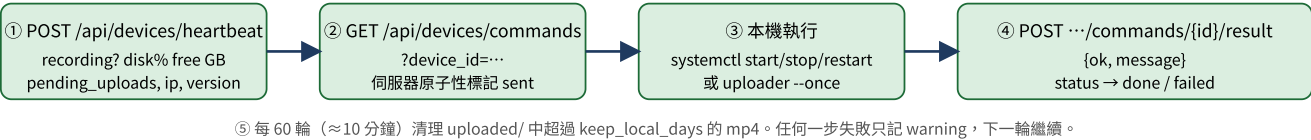


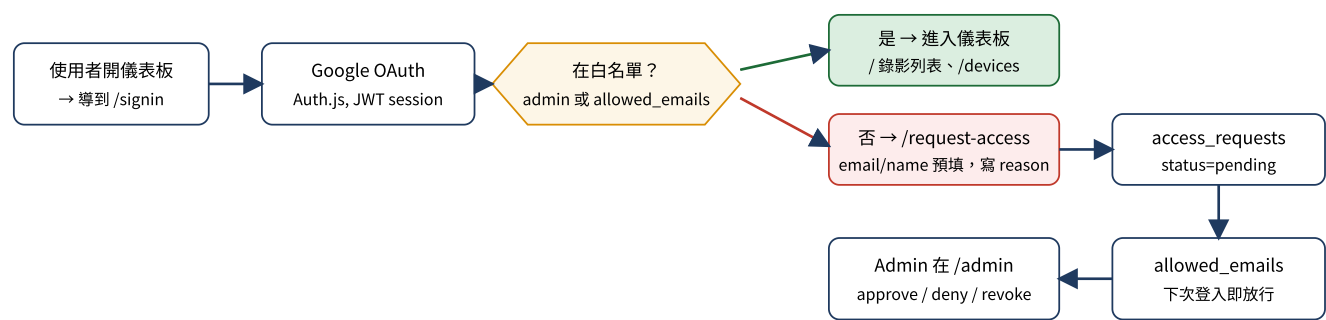
圖 3 現有兩條流程。指令由管理者在儀表板 /devices 頁按下後寫入 device_commands (pending)，最慢 10 秒內被 Pi 拉走。

設定重點（`pi/config/vigil.yaml`）

區塊	關鍵欄位	說明
device	id / label / location	id 必須等於 Pi hostname（如 <code>vigil-hsinchu-01</code> ），它同時是 R2 key 前綴與儀表板上的裝置 ID。
camera	device_macos "0" / device_linux /dev/video0 / 1280x720 / 30fps	同一份設定 Mac 與 Pi 都能跑。
recording	segment_seconds 600 / codec libx264 / preset veryfast	Pi 5 沒有 H.264 硬體編碼器，A76 核心軟編 1080p30 沒問題。
storage	keep_local_days 3 / max_disk_usage_percent 85	agent 的清理依據。
upload	bucket vigil-recordings / scan 30s / stability_grace 30s / delete_after_upload false	金鑰放 <code>pi/.env</code> ，不進 git。
agent	heartbeat 10s / cleanup_every 60 輪	需要 <code>VIGIL_DASHBOARD_URL</code> 、 <code>VIGIL_DEVICE_API_KEY</code> 。

6. 雲端儀表板：登入授權流程與資料模型

web/ 是 Next.js 16 (App Router) + Auth.js v5 + Drizzle ORM 。路由守門在 src/proxy.ts : /signin 、 /request-access 、 /api/auth 、 /api/devices/* 公開 (後者用 bearer token) ; 其他頁面要 session ; /admin 只有 ADMIN_EMAIL 環境變數列出的信箱能進。



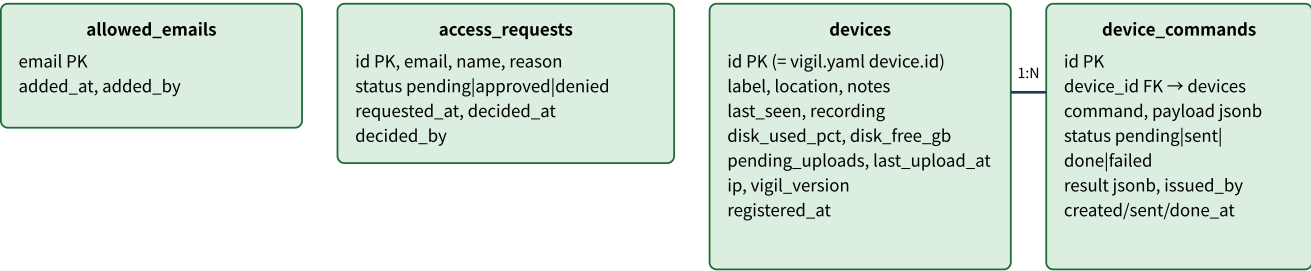
Pi 走另一條路 : /api/devices/* 用 Authorization: Bearer VIGIL_DEVICE_API_KEY (全機隊共用一把, 比對 Vercel 環境變數)

圖 4 人類使用者的登入 / 申請 / 核准流程。

頁面與 API

路徑	誰用	功能
/	已登入使用者	錄影列表 (從 R2 ListObjects 解析 key → 裝置 / 日期 / 時間 / 大小), 預覽或下載 (presigned GET, 1 小時)
/devices	已登入使用者	裝置清單 (最後心跳、錄影中、磁碟、待上傳) 與指令按鈕 → 寫入 device_commands
/admin	admin	審核 access_requests、管理 allowed_emails
/signin、/request-access	公開	登入頁、申請存取頁
POST /api/devices/heartbeat	Pi (bearer)	upsert devices : label/location 只在有值時覆蓋, 狀態欄位每次覆蓋
GET /api/devices/commands?device_id=	Pi (bearer)	取 pending 指令並原子性標為 sent, 避免重複派送
POST /api/devices/commands/{id}/result	Pi (bearer)	回寫 result JSON、status done/failed、doneAt

現有資料模型 (4 張表)



影片本身不在資料庫：R2 的 object key 就是索引（<device>/YYYY/MM/DD/vigil_日期_時間.mp4）。

圖 5 現有 Postgres (Neon) schema，由 Drizzle 管理（`npm run db:push`）。

7. 為什麼先做光固化 3D 列印、光學限制

痛點容易量化，所以好賣也好誠實評估

一次樹脂列印無人看顧跑 4–12 小時。最主要的失敗——物件從平台脫落——通常發生在前 10%，而印表機完全不知道。它會繼續循環剩下的十小時，什麼都沒固化，同時：槽裡的樹脂被消耗並部分固化成一坨；那坨東西可能戳破 FEP 膜（耗材，破了就是樹脂外洩）；機時全部浪費；交期整整延一輪，因為要到早上才有人發現。在一層之內（約十秒）抓到，一夜報廢變成二十分鐘的重來。盒子有沒有回本，沒有任何模糊空間。

物理限制——決定成敗的部分

樹脂列印對相機非常不友善。把問題說精確，因為機構與打光設計直接由此推出：

問題	內容
幾何	MSLA 的物件是從平台 向下 長的：每層在物件底部、貼著槽底 FEP 固化，然後平台抬升一層厚度。所以物件倒吊在平台下方，露出樹脂面的比例隨列印進行而 增加 。前面幾層——正是最容易脫板的時候——最難看見，因為物件還泡在樹脂面附近。這是整個問題最彆扭的事實，第 8 章的檢測階梯就是為了繞過它。
對比	物件濕漉漉、半透明、通常灰或黑，吊在同樣灰或黑的樹脂槽上方。前景與背景幾乎沒有區別。
光	內部很暗，但不能直接加白光 LED：藍光成分會在長時間列印中慢慢固化槽裡的樹脂表面。照明必須在 ~600 nm 以上——深紅（620–660 nm）或近紅外（850 nm），光敏樹脂對這些波長基本上是瞎的。
罩子反而是資產	橘色抗 UV 罩的功能是擋短波長，所以它對我們要用的紅光 / 近紅外高度透明。相機不用住在罩內的煙氣與熱裡，也不用每次操作員開蓋就重新對準： 裝在罩外、從罩外打光、隔著罩拍 。代價是壓克力反射，用離軸安裝角度處理，必要時加偏光片。

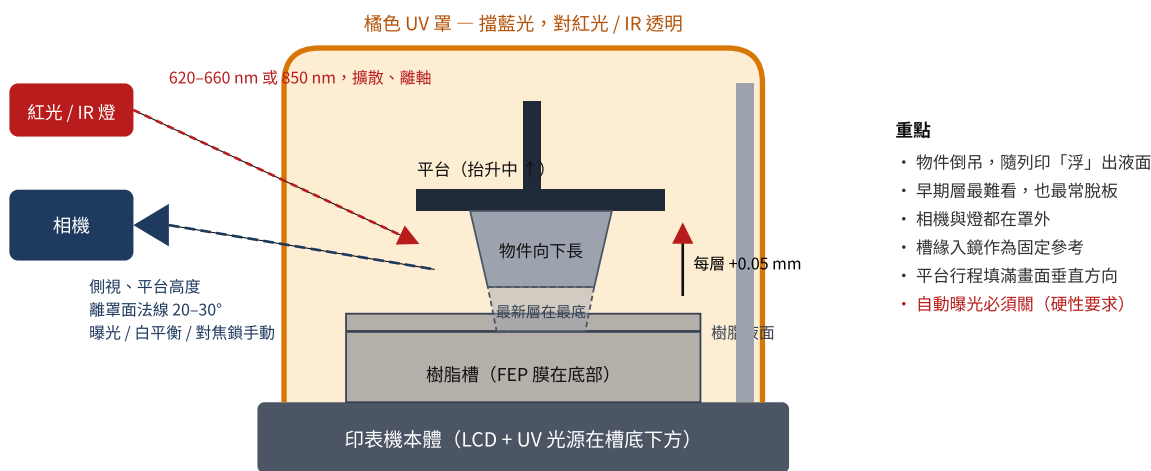


圖 6 光學配置示意。對只看過 FDM 的人：這裡的物件是往下長、從樹脂裡浮出來的。

建議的光學配置

v1 (沿用平台已支援的 USB webcam)	產品化
--------------------------	-----

相機	UVC webcam， 曝光 / 白平衡 / 對焦鎖定為手動	Pi Camera Module 3 NoIR
光源	620–660 nm 紅光 LED 條，擴散、離軸	850 nm IR 照明 —— 操作員看不見
位置	側視、平台高度、離罩面法線 ~20–30°	同上，磁吸底座關節臂
構圖	平台行程垂直填滿畫面；槽緣入鏡作固定參考	同上

自動曝光必須關閉。會隨物件變大而重新曝光的相機，會毀掉第 8 章一切依賴的幀對幀比較。這是硬性要求，不是調校偏好。

8. 核心洞見與檢測階梯 L0-L4

樹脂印表機是節拍器

每一層它做同樣的動作：抬升平台、停留、下降、曝光、重複 —— 幾千次，機械重複性極高。這給了通用監控相機永遠拿不到的東西：如果我們**每個週期在同一運動相位取一幀** —— 在抬升頂點、物件滴乾靜止之後 —— 得到的影像堆疊**已經對齊**。不用對位、不用姿態估計。第 842 幀與第 843 幀只差 0.05 mm 一層，等於沒差。任何有意義的幀間差異都是訊號，雜訊底由滴液與反光決定，而不是相機移動。

- **週期只從影片偵測**。我們觀察週期性運動特徵並鎖上，從不問印表機。這守住「夾上任何機器」的承諾，也直接推廣到其他週期性機器（第 15 章）。
- **每層幀堆疊 = 免費的逐層縮時影片** —— 使用者本來就想要的功能，零成本從管線掉出來。

檢測階梯（依此順序建置，每一級可獨立出貨）

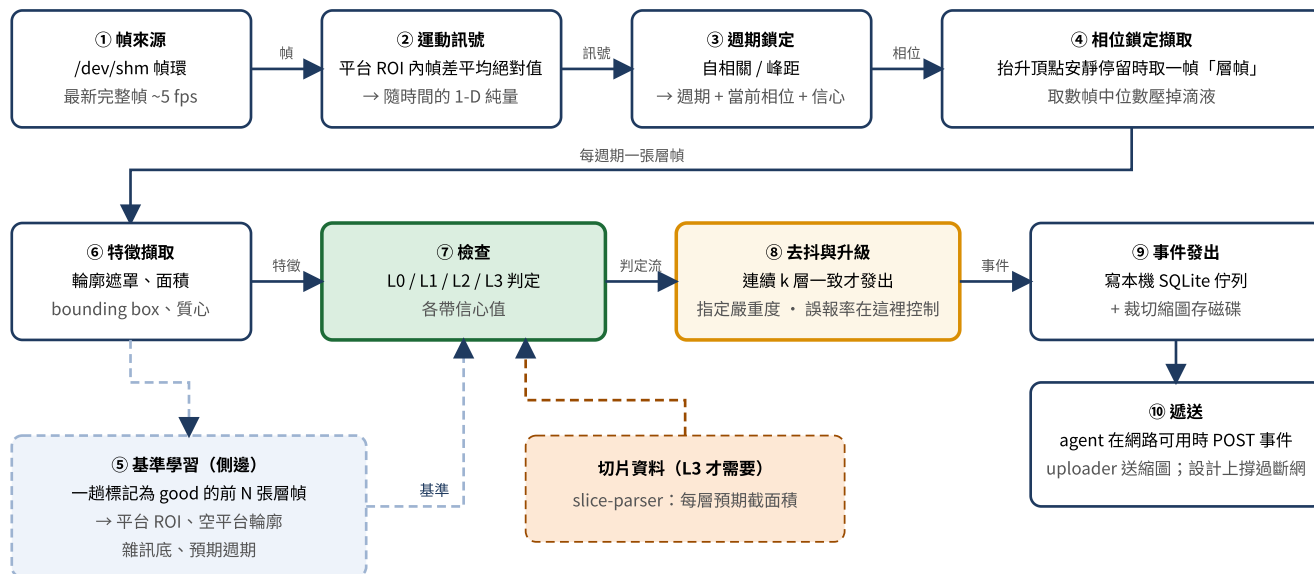
L4	雲端模型讀可疑幀 失敗 vs 反光、失敗分類、白話解釋 · 雲端 · 每次列印幾次呼叫	讓凌晨三點的人信任它
L3	觀測輪廓 vs 切片檔的預期截面 印錯檔、缺區域、LCD 壞點 · 邊緣 + 切片解析 · ms/層	真正的護城河：我們有切片檔
L2	第 N 幀 vs N-1：輪廓是否單調成長？ 分層、支撐失敗、部分掉落 · 邊緣 OpenCV · ms/層	開始感覺聰明
L1	平台上還吊著東西嗎？（輪廓 vs 學到的空平台基準） 脫板 —— #1 失敗 · 邊緣 OpenCV · ms/層	L1 就是產品
L0	機器還在以預期週期循環嗎？ 結束、卡住、當機、斷電、中途停 · 邊緣 · 幾乎零成本	一天的工，立刻有用

圖 7 檢測階梯。L0-L3 在邊緣跑幾何運算，不需要模型；只有 L4 上雲。

關於第 7 章的早期層問題：即使物件太靠近液面無法乾淨分割，脫板仍可偵測 —— 脫板之後，輪廓**停止成長**而平台持續上升。單調成長是比直接分割弱的訊號，但它優雅退化，並且正好涵蓋分割最差的那個時間窗。所以 L1 與 L2 是互補，不是重複。

9. 檢測管線流程圖（inspector 十個階段）

這是 `inspector` 內部的主要資料流。每個階段的輸出是下一個的輸入；第 5 階段的基準從側邊餵入第 7 階段，不在主線上。



給實作者的備註

- ⑧ 是實際控制誤報率的地方，也是最可能需要對真實影片調參的階段。
- ⑤ 是每次安裝都必須「學」而不是出廠帶固定閾值的原因 —— 每次的印表機、罩子、桌面、角度都不同。
- ROI、閾值、去抖 k 存在雲端 `devices.inspection_config` (jsonb)，調參不需要對機隊部署程式碼。
- ①~⑨ 全在邊緣、無網路也能跑；⑩ 把「有網路」變成純遞送問題。
- 幀率刻意低 (~5 fps)：③ 要解析的是數秒的抬升行程。
- 每週期取一張層幀後，影像堆疊天生已對齊 —— 不需要任何對位或姿態估計（第 8 章）。
- 相機被撞歪、遮住、鎖相失敗 → 不是報錯，是進入 LOST 狀態（第 10 章）。

圖 8 inspector 十階段資料流：幀 → 純量訊號 → 相位 → 層幀 → 特徵 → 判定 → 事件。

10. 列印工作狀態機

一個 **job** 就是一趟列印。邊緣端從週期推斷工作邊界 —— 沒有人告訴它列印何時開始。LOST 是一級狀態，不是錯誤：有人撞到相機之後還默默產生垃圾的盒子，比一個會說「我看不到了」的盒子更糟。

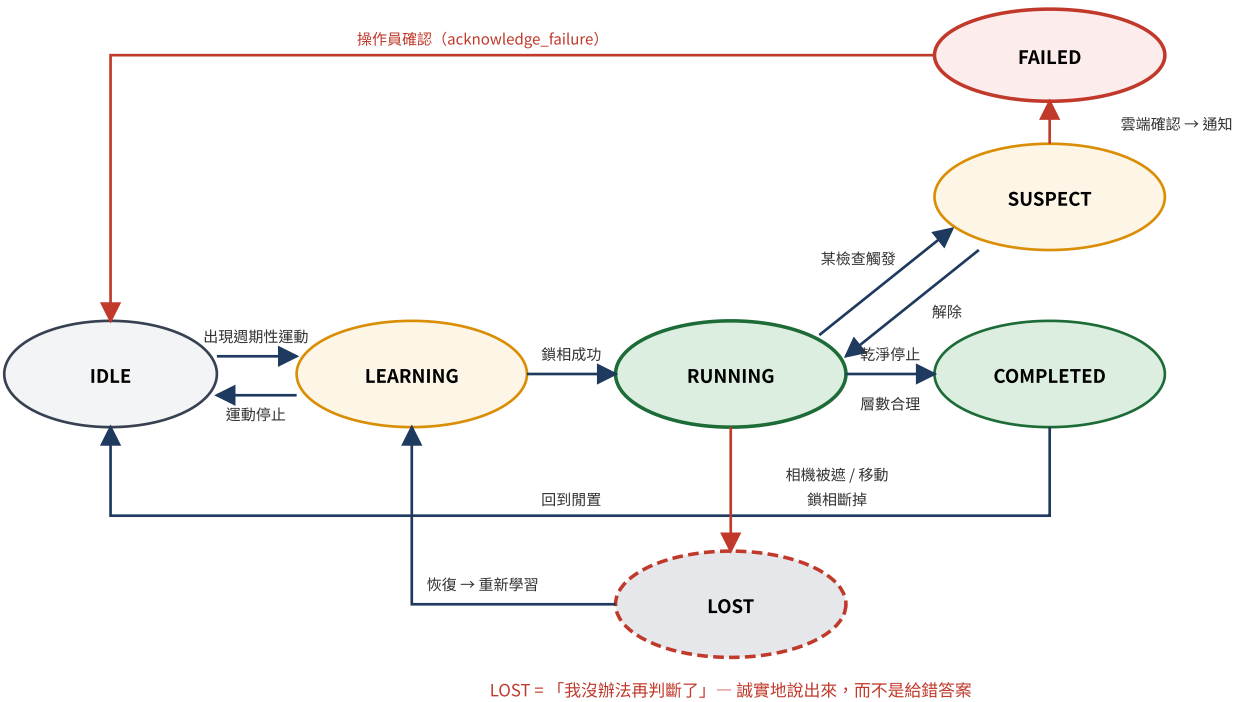


圖 9 工作狀態機。LOST 與快樂路徑同等重要。

狀態	意義	轉出
IDLE	沒偵測到週期。機器關機或已完成。	出現週期性運動 → LEARNING
LEARNING	偵測到週期，正在取得週期與基準。	鎖定 → RUNNING；運動停止 → IDLE
RUNNING	已鎖定，檢查啟動，層數計數中。	SUSPECT、COMPLETED、LOST
SUSPECT	某檢查觸發；等待去抖與雲端確認。	解除 → RUNNING；確認 → FAILED
FAILED	確認失敗。已通知。	操作員確認 → IDLE
COMPLETED	在合理層數後乾淨停止。	IDLE
LOST	相機被遮、移動、或鎖相斷掉 —— 我們無法再判斷。	恢復 → LEARNING

11. 失敗升級時序圖

從邊緣判定到手機通知的完整路徑。注意 L4 呼叫的方向：**Pi 永遠不和模型講話**。Pi 把事件 POST 到我們的 API，由 API 決定是否值得呼叫模型——模型金鑰集中在一處，每趟列印的雲端成本可在伺服器端強制控管。

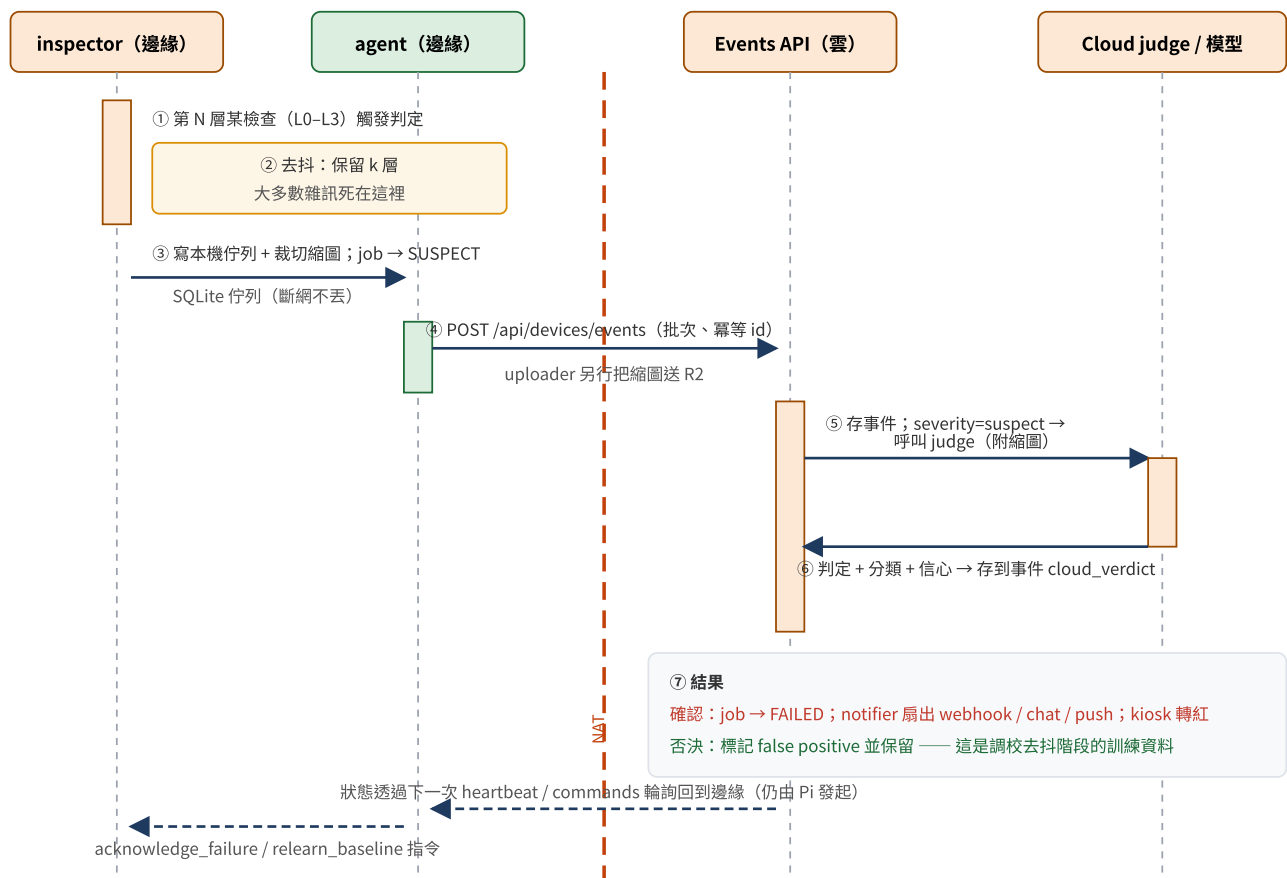


圖 10 失敗升級時序：邊緣判定 → 去抖 → 佇列 → API → 雲端判讀 → 通知。所有穿越 NAT 的箭頭仍由邊緣發起。

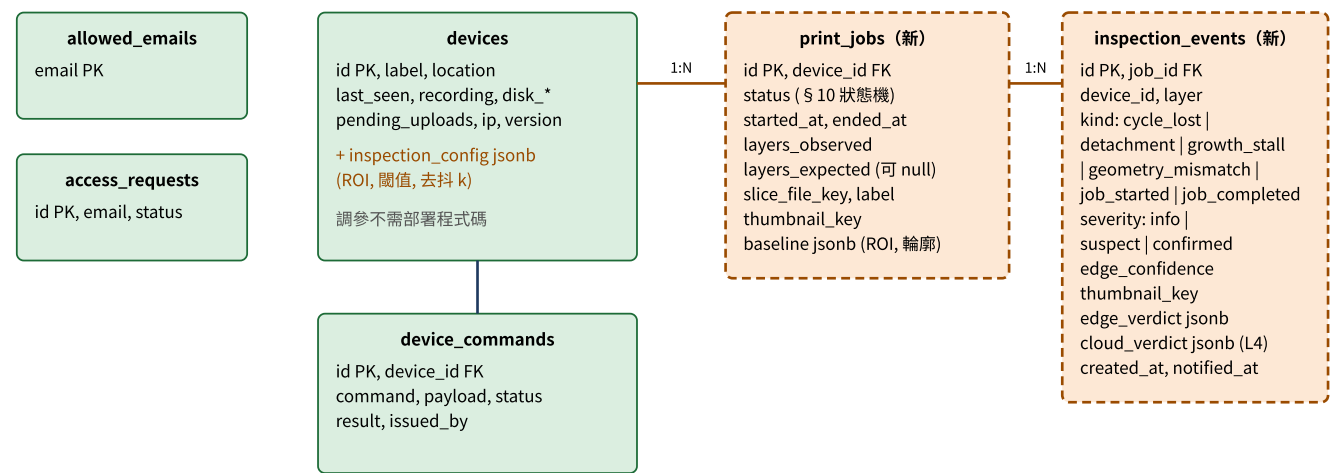
12. API 與資料模型變更

裝置端 API (bearer token，沿用現有機制)

方法	路徑	狀態	用途
POST	/api/devices/heartbeat	存在 擴充	裝置健康；新增 job 狀態、當前層數、鎖相信心
GET	/api/devices/commands	存在	拉取排隊指令
POST	/api/devices/commands/{id}/result	存在	回報指令結果
POST	/api/devices/events	新	提交檢測事件（批次；以客戶端提供的 id 冪等——邊緣可能在斷網後重送）
POST	/api/devices/jobs	新	開 / 關工作、回報狀態轉換

使用者端新路由：`/jobs/{id}`（每趟列印的時間線：縮圖、層數、判定、逐層縮時）與 `/kiosk/{device_id}`（機台旁顯示）。指令佇列新增：`mark_job_good`（觸發基準學習）、`relearn_baseline`、`acknowledge_failure`，以及僅 Phase 5 的 `abort_print`。

資料模型 ERD (現有 + 新增)



綠色 = 現有 (schema.ts 中)；橘色虛線 = PRODUCT.md § 6.7 規劃新增。縮圖與影片仍在 R2，表裡存 key。

圖 11 資料模型：現有四張表與新增的 print_jobs / inspection_events。

13. 機台旁的螢幕（kiosk）

做成儀表板 app 裡的一條 route，Pi 的 HDMI 輸出跑 Chromium 全螢幕指向該 URL。不需要新硬體、不需要第二套程式碼、不需要顯示驅動；同一個 URL 在壁掛平板或操作員手機上也能用。

它必須誠實地退化：本機快取最後已知狀態，斷網時由 Pi 提供 fallback 頁，顯示本機狀態並明說「目前離線」。停在過期好消息上的狀態螢幕比沒有螢幕更糟。

優先序	內容
1	當前狀態：一整塊大色塊，隔一個房間也看得懂（綠 / 黃 / 紅 / 灰）
2	層數、已用 / 剩餘時間
3	即時畫面
4	最近幾筆事件
之後	由 Pi GPIO 驅動的實體燈塔，給想從工廠另一頭讀狀態的人

14. 階段路線圖與驗收條件

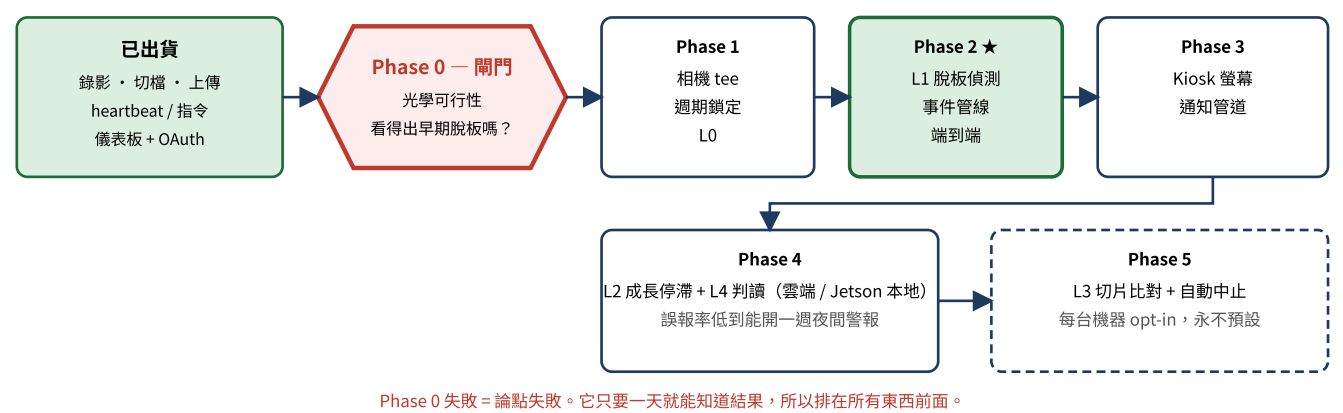


圖 12 路線圖。Phase 0 是所有後續工作的關門。

#	目標	驗收條件
0	光學可行性。手機或 webcam + 紅光，一趟真實列印，錄完整趟 —— 包含一次刻意製造的失敗。	能用肉眼從影片指出失敗開始的那一層 —— 包含前 10% 層數的失敗。做不到，就先改機構 / 打光再寫任何程式。
1	相機 tee + 週期鎖定 + L0。	從影片推算的層數與印表機自己的計數在整趟列印內差 ±2 內。機器停止在一個週期內回報。幀環驗證無撕裂。
2	L1 脫板偵測 + 事件管線端到端。	破壞性列印在 3 層內發出事件，儀表板可見含縮圖。3 趟乾淨列印零誤報。列印中斷網事件不丟。
3	Kiosk 螢幕 + 通知管道。	操作員在機台旁不用筆電就看到即時狀態，含誠實的離線狀態。失敗一分鐘內到手機。
4	L2 + L4 判讀 —— 基本版在雲端，Jetson 版在盒內本地跑。	失敗事件帶分類與白話解釋。誤報率低到可以連續一週開著夜間警報。
5	L3 切片比對 + 自動中止。	盒子能停掉註定失敗的列印，不只是回報。

關於自動中止（每個人第一個問的功能）：停掉列印是剩餘價值最大的地方，也正是「任何機器都能用」承諾破掉的地方。較新的 Elegoo / Anycubic 有區網協定；Chitu 板子大多沒有。通用備案 —— 智慧插座斷電 —— 很粗暴，平台會停在槽裡，不是優雅中止；它只能讓浪費與 FEP 風險不再擴大。必須每台機器 opt-in，永不預設。先出貨偵測，再贏得行動的權利。

15. 風險與明確不做的事

風險（最糟的排前面）

#	風險	緩解
1	影像分不開。灰物件、灰樹脂、暗櫃子。	Phase 0 失敗 = 論點失敗。唯一的緩解是：只要一天就知道，而且排在所有事之前。
2	透明 / 半透明樹脂明顯比填充灰難。	可能需要不同角度、背光、或 v1 明列不支援材料清單。
3	每次安裝都不同 —— 印表機、罩、桌面、角度。	基準必須每次安裝學，永不出廠帶。重新學習必須一鍵。
4	夜間誤報對信任是致命的。一次凌晨三點的假警報，操作員就永久關掉通知。	這是去抖階段與 L4 存在的全部理由：等第二意見同意再發警報。
5	盒子要活過工廠 —— 樹脂煙氣、IPA、灰塵、有人撞到相機臂。	機構剛性是產品需求。被移動的相機要被偵測（LOST 狀態），而不是默默產生垃圾。
6	不熟悉機器的週期鎖定。鎖定只在我們擁有的印表機上驗證過。	LEARNING / LOST 狀態讓它成為可見、誠實的失敗，而不是錯誤答案。

明確不做

- 零件級尺寸檢測（AOI）。不同產品、不同價格、不同銷售方式。
- v1 任何需要印表機配合的東西 —— 不整合控制器、不碰韌體、不接線。
- 針對物件訓練的模型。L0-L3 是幾何運算不是學習，所以邊緣永遠不需要推模型過去，盒子在沒見過的物件第一趟列印就能工作。
- 多相機。一台相機、一台機器、一個盒子。

推廣性

第 8-11 章沒有任何東西是樹脂專屬的。盒子從影片鎖定週期性機器的節奏、學習好的週期長什麼樣、回報偏差 —— 對射出成型機、跑重複程式的 CNC、充填 / 貼標線、pick-and-place 供料器，都是同一個產品。只有三件事是樹脂專屬：照明波長限制（§ 7）、L3 切片解析（§ 8）、失敗分類（§ 12）。週期鎖定、相位鎖定取樣、基準學習、去抖、狀態機、整個雲端與回報路徑都原封不動。樹脂列印是灘頭堡，因為失敗夠戲劇化、漏掉的成本容易算、機器便宜到能拿來實驗、買家找得到。

16. 開發上手指南

你需要拿到的東西（向 Maintainer 要）

項目	用途
GitHub repo 存取權 (MaxShih147/vigil)	程式碼
pi/.env : VIGIL_S3_ENDPOINT / ACCESS_KEY_ID / SECRET_ACCESS_KEY、VIGIL_DASHBOARD_URL、VIGIL_DEVICE_API_KEY	邊緣端上傳 R2、連儀表板（建議：開發者各自申請一組 R2 token，只給 vigil-recordings 讀寫）
web/.env.local : R2 四項、DATABASE_URL(_UNPOOLED)、AUTH_SECRET、AUTH_TRUST_HOST、ADMIN_EMAIL、AUTH_GOOGLE_ID/SECRET、VIGIL_DEVICE_API_KEY	本機跑儀表板（只做邊緣端的人不需要）
儀表板白名單（請 admin 在 /admin 核准你的 Google 帳號）	看裝置與錄影
Tailscale tailnet 邀請	從任何地方 SSH 進 Pi

Mac 上開發邊緣端

```
cd pi
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt # PyYAML, boto3 (另需 brew install ffmpeg)
cp .env.example .env && $EDITOR .env

./scripts/start_recording.sh --print-cmd # 只印 ffmpeg 指令，不錄
./scripts/start_recording.sh # 第一次會跳相機權限，允許後重跑
./scripts/start_uploader.sh --once # 驗證 R2 認證 + 掃一次
./scripts/start_uploader.sh # 常駐
.venv/bin/python scripts/agent.py --once # 一次 heartbeat，期望 "heartbeat ok ..."
```

錄影落在 pi/data/recordings/，每 segment_seconds 一檔；穩定 30 秒後上傳，成功搬到 pi/data/uploaded/。

本機跑儀表板

```
cd web
npm install
npm run db:push # Drizzle 把 schema 推到 Neon (改 schema.ts 後也是這個)
npm run dev # http://localhost:3000
```

web/AGENTS.md 提醒：這版 Next.js (16.x) 與多數人熟悉的版本有 breaking changes，寫程式前先讀 node_modules/next/dist/docs/。

Pi 5 部署（摘要，完整 runbook 見 pi/DEPLOY.md）

步驟	內容
硬體	Pi 5 (4GB+)、 5V/5A USB-C 電源 (3A 會讓 USB 週邊掉電)、32GB+ SD 或 USB SSD、UVC webcam (C270 / C920 可靠)
1-3	Pi Imager 燒 Raspberry Pi OS Lite 64-bit；hostname <code>vigil-<地點>-<NN></code> (= device.id)；使用者 <code>vigilop</code> ；SSH 只允許公鑰；首次 Wi-Fi 用 iPhone 熱點最穩 (企業 Wi-Fi 常有 AP isolation)
4	立刻裝 Tailscale (<code>tailscale up --ssh</code>)。之後不管 Pi 接哪個 Wi-Fi 都能連。
5-6	用 nmcli 多加幾組 Wi-Fi；dev Pi 可設 passwordless sudo
7-9	clone 到 <code>~/vigil</code> 、建 <code>venv (--system-site-packages)</code> 、 <code>scp .env</code> 、改 <code>vigil.yaml</code> 的 device.id / label / location、camera.device_linux、codec libx264
10	依序 smoke test： <code>agent --once</code> → 儀表板 /devices 看到裝置； <code>boto3 list R2</code> ； <code>ffmpeg</code> 錄 5 秒；跑 12 分鐘看一個段被上傳
11	正式機：repo 放 <code>/opt/vigil/</code> 、專用 <code>vigil</code> 使用者、設定放 <code>/etc/vigil/vigil.yaml</code> 與 <code>/etc/vigil/.env</code> 、安裝三個 systemd unit 並 <code>enable --now</code>

Pi 5 陷阱：沒有 H.264 硬體編碼器 (`h264_v4l2m2m` 不存在，用 `libx264`)；`/dev/video19-35` 是內建 ISP / HEVC 解碼器不是相機，webcam 是 `/dev/video0`；關機後紅燈仍亮是正常；micro-HDMI；風扇 50°C 以下不轉。

17. 文件審閱：發現的不一致與交接前該處理的事

整體而言文件品質很高：`PRODUCT.md` 自給自足、決策附理由、附錄 A 直接列出該畫哪些圖；`DEPLOY.md` 是實戰過的 runbook。以下是我對照程式碼後發現、建議在分享給同事前處理的點。

A. 安全 —— 分享前必做

#	發現	建議
1	資料夾內有 <code>pi/.env</code> (6 行) 與 <code>web/.env.local</code> (17 行) 真實金鑰：R2 secret、Neon 連線字串、 <code>AUTH_SECRET</code> 、Google OAuth secret、裝置 API key。它們已 gitignored，但若直接複製資料夾 / zip 分享就會外流。	只透過 git 分享；金鑰用密碼管理工具另外給。考慮為每位開發者開獨立的 R2 token，方便撤銷。
2	<code>VIGIL_DEVICE_API_KEY</code> 是全機隊共用一把 bearer token，任何拿到它的人可以冒充任何裝置寫 heartbeat / 拉走指令。	目前 pilot 階段可接受；Phase 2 事件 API 上線前建議改為每裝置一把 key (devices 表加 <code>api_key_hash</code>)，輪替不影響整隊。
3	<code>.claude/settings.local.json</code> 與 <code>web/.vercel/</code> 是個人 / 環境設定。	確認在 <code>.gitignore</code> 內 (<code>.vercel</code> 已在； <code>.claude/settings.local.json</code> 請確認)。

B. 文件與程式碼不一致

#	發現	建議
4	<code>README.md</code> Pi deployment 只 enable <code>vigil-recorder</code> 與 <code>vigil-uploader</code> ，但 <code>pi/systemd/</code> 有三個 unit， <code>vigil-agent</code> 沒裝就不會出現在儀表板。	<code>README</code> 加上 <code>vigil-agent</code> 。
5	<code>vigil.yaml</code> 註解寫「Pi 上改成 <code>h264_v4l2m2m</code> 硬體編碼」，但 <code>DEPLOY.md</code> 明說 Pi 5 已移除該編碼器。	改註解為「Pi 4 才有 <code>h264_v4l2m2m</code> ；Pi 5 用 <code>libx264</code> 」。
6	階段編號兩套： <code>README</code> 的 Inspection 用 0、6、7、8、9； <code>PRODUCT.md</code> § 7 用 0-5。同事會混淆。	統一成一套 (建議以 <code>PRODUCT.md</code> 的 0-5 為準， <code>README</code> 改成對照)。
7	<code>README</code> 說 <code>pi/config/vigil.yaml</code> 預設值是 dev Mac (<code>device.id</code> <code>vigil-dev</code>)。若同事直接在 Pi 上跑而沒改，會與其他人的 dev 裝置在儀表板上互相覆蓋 (heartbeat 是 upsert by id)。	<code>DEPLOY.md</code> 已提醒；建議 agent 啟動時若 <code>device.id</code> 與 <code>hostname</code> 不符就印 warning。
8	<code>agent.py</code> 的 <code>vigil_version</code> 寫死 "0.1.0"。	改讀 git short hash 或 <code>VERSION</code> 檔，機隊升級才追得到。
9	<code>start_recording</code> / <code>stop_recording</code> 等指令在 Mac 或沒 <code>systemd</code> 的 dev Pi 上只會回「would start …」，不會真的動作。	文件內註明這些指令只在 <code>systemd</code> 部署下有效。

C. PRODUCT.md 內部值得補的地方

#	發現	建議
---	----	----

10	§ 6.3 的 tmpfs 幀環「無撕裂讀取」與 5 fps 是否足夠，列為 Phase 1 待確認，但沒有寫驗證方法。	加一句：例如 ffmpeg 用 <code>-f image2 -atomic_writing 1</code> 或先寫暫存名再 rename；inspector 只讀編號最大減一的檔。
11	L1 需要「空平台基準」，但流程上何時能看到空平台（列印開始前）與如何觸發 <code>mark_job_good</code> 未定義；job 邊界又是從週期推斷的。	Phase 2 設計時明確：基準是在 LEARNING 狀態自動取「前 N 層」還是要人工標記；兩者對 UX 差很多。
12	通知管道（webhook / chat / push）沒指定第一個實作目標。	建議先 LINE Notify 已停服 → 用 LINE Messaging API 或 Telegram bot / Slack webhook 擇一，Phase 3 才不用再開會。
13	儀表板 UI 目前是「Matrix / 磷光綠 CRT 終端機」風格（commit 91ee5d4），與 kiosk「三公尺外可讀的大色塊」需求可能衝突。	kiosk route 獨立設計，不沿用儀表板主題。

D. 研究軌與硬體分級

研究計畫見 `docs/RESEARCH.md`：四個研究問題（週期鎖定、脫板偵測延遲、幾何式 vs 學習式的跨安裝泛化、本地判讀 vs 雲端判讀）、失敗誘發協定、以韌體層數與 Z 軸電流為 ground truth、36 趟資料集、投稿目標。硬體分兩級（`PRODUCT.md` § 6.10）：Pi 5 為基本版跑 L0–L3；Jetson Orin Nano 為判讀版，在盒內跑 L4 視覺模型（影像不出廠），同時是研究用的基線實驗台。

E. 建議的分工切點（供你安排人力參考）

工作包	相依	技能
Phase 0 光學實驗：治具、紅光、透過罩子拍一趟含刻意失敗的列印	無 —— 立刻可做	機構 / 光學 / 對印表機熟
ffmpeg tee + /dev/shm 幀環 + 撕裂驗證	無	Linux / ffmpeg
週期鎖定 + L0 + 狀態機（可先用 Phase 0 的錄影離線開發）	Phase 0 影片	Python / OpenCV / 訊號處理
Events API、 <code>print_jobs</code> / <code>inspection_events</code> schema、 <code>/jobs</code> 時間線 UI	無（可與邊緣並行，先定 API 合約）	Next.js / Drizzle / Postgres
Kiosk route + 本機 fallback + Chromium kiosk 設定	Events API	前端 / Pi 系統
Jetson Orin Nano 研究台：學習式基線（CNN / PatchCore）+ 本地 L4 判讀	Phase 0 影片、資料集	PyTorch / JetPack / 小型 VLM 部署